

# Gradient descent and SGD

From a first-order Taylor model to the update rule and the learning rate, then cheaper steps from one example or a small batch.

## THE QUESTION

Most losses have no closed-form minimiser. How do we walk downhill to a good  $\theta$ , and how much data should each step look at?

## THE INTUITION

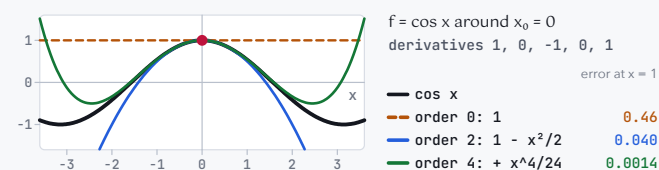
Near the current point the loss is almost a plane, and a plane falls fastest along  $-\nabla L$ . Take a small step that way and repeat. One random example's gradient points the right way on average, so SGD steps are  $n$  times cheaper but noisy.

## KEY TAKEAWAY

$\theta \leftarrow \theta - \alpha \nabla L(\theta)$ . The learning rate  $\alpha$  sets speed and stability: too small crawls, too large bounces or diverges. Batch, mini-batch and SGD differ only in how many examples estimate  $\nabla L$ .

current  $\theta$  → gradient  $g$  at  $\theta$  → step  $\theta - \alpha g$  → watch the loss → stop: small  $g$  or max steps

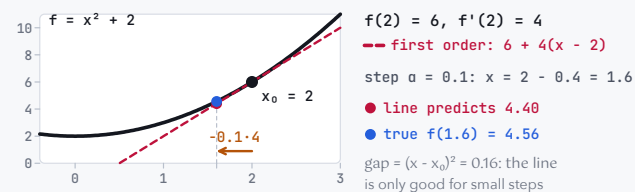
### 1 A Taylor series matches $f$ near $x_0$



$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2 + \dots$$

Order 1 is the tangent line, order 2 a parabola. For  $\cos x$  at 0 the derivatives cycle, giving  $1 - x^2/2 + x^4/24$ . More terms fit better near  $x_0$ , yet all drift far from it: at  $x = 2$  order 4 is off by 0.08.

### 2 Step against the gradient



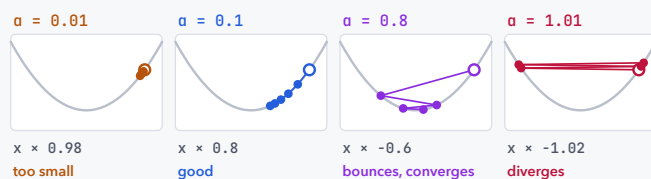
In  $n$  dimensions  $f(x_0 + \Delta x) \approx f(x_0) + \nabla f^T \Delta x$  (the next term uses the Hessian). The change  $\nabla f^T \Delta x = \|\nabla f\| \|\Delta x\| \cos \varphi$  is most negative at  $\varphi = 180^\circ$ , so step  $\Delta x = -\alpha \nabla f$  with  $\alpha > 0$ : steepest descent, straight across the contours (see the maths for ML sheet). The line ignores curvature, so keep steps small.

### 3 Repeat until a stopping rule fires

Stop when  $\|g\|$ , the change in loss or the change in  $\theta$  drops below  $\epsilon$ , and always cap the steps. With fixed  $\alpha$  the steps shrink by themselves as  $\|g\| \rightarrow 0$ . GD finds a local minimum; the global one for a convex loss and a small enough  $\alpha$ . On the three points of panel 5:

```
X = np.array([[1, 1], [1, 2], [1, 3]])
y = np.array([1, 2, 3])
theta, alpha = np.array([4., 0.]), 0.1
for t in range(10_000):
    g = 2 / 3 * X.T @ (X @ theta - y)
    if np.linalg.norm(g) < 1e-6: break
    theta = theta - alpha * g
print(t, theta.round(4)) # 567 [0. 1.]
```

### 4 The learning rate sets speed and stability



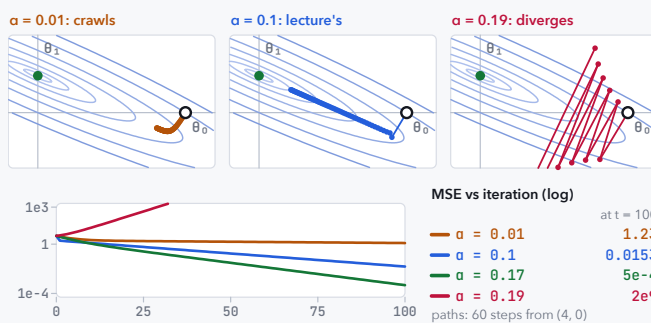
On  $f = x^2 + 2$  from  $x_0 = 2$ ,  $x_{t+1} = (1 - 2\alpha)x_t$ :  $\alpha = 0.01$  keeps 98% of the distance per step, 0.1 keeps 80%, 0.8 overshoots but shrinks, 1.01 grows 2% per step. An exploding loss means  $\alpha$  is too large, slow progress too small; for oscillation use a smaller  $\alpha$  or momentum.

### 5 Worked: a line through (1, 1), (2, 2), (3, 3)

| t   | $\theta_0$ | $\theta_1$ | MSE   |
|-----|------------|------------|-------|
| 0   | 4.00       | 0.00       | 4.667 |
| 1   | 3.60       | -0.67      | 1.923 |
| 3   | 3.46       | -0.52      | 1.716 |
| 10  | 2.92       | -0.28      | 1.220 |
| 100 | 0.33       | 0.86       | 0.015 |

With  $\epsilon_i = y_i - \hat{y}_i$ :  $\partial L / \partial \theta_0 = -\frac{2}{n} \sum \epsilon_i$  and  $\partial L / \partial \theta_1 = -\frac{2}{n} \sum \epsilon_i x_i$ , or  $\nabla L = \frac{2}{n} X^T (X\theta - y)$ . From (4, 0) the residuals are -3, -2, -1, the gradient (4, 6.67), and  $\alpha = 0.1$  steps to (3.6, -0.67).

### 6 $\alpha$ must stay below $2/\lambda_{\max}$



The Hessian  $\frac{2}{n} X^T X = \begin{pmatrix} 2 & 4 \\ 4 & 9.33 \end{pmatrix}$  has eigenvalues 11.09 and 0.24. Along each eigenvector the error is multiplied by  $1 - \alpha \lambda$  per step, so GD converges only for  $\alpha < 2/\lambda_{\max} = 0.18$ . The flat direction sets the pace; 0.17 zigzags yet is fastest. Standardised features give rounder contours.

# One example gives an unbiased, noisy gradient

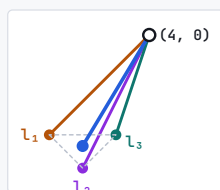
The full gradient is an average over examples, so a random subset estimates it: cheaper steps on a noisy path.

## 7 Three variants differ in batch size

| one update uses                                          | per update | updates/epoch |
|----------------------------------------------------------|------------|---------------|
| batch<br>12 examples, shuffled each epoch (illustrative) | all $n$    | 1             |
| mini-batch<br>32 to 256                                  | $b$        | $n / b$       |
| SGD<br>1                                                 | 1          | $n$           |

An epoch is one pass over the data; mini-batches of 32 to 256 are usual and suit GPUs. The lecture's SGD run from  $(4, 0)$ ,  $\alpha = 0.1$ : point  $(1, 1)$  has  $\varepsilon = -3$ , gradient  $(-2\varepsilon, -2\varepsilon x) = (6, 6)$ , so  $(3.4, -0.6)$ ; then  $(2, 2)$  has  $\varepsilon = -0.2$ , gradient  $(0.4, 0.8)$ , so  $(3.36, -0.68)$ . A batch step goes to  $(3.6, -0.67)$  instead.

## 8 $\nabla L$ is the mean of per-example gradients



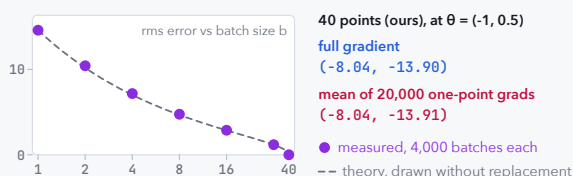
gradients at  $\theta = (4, 0)$ , three points  
 point  $(1, 1)$ : grad  $l_1 = (6, 6)$   
 point  $(2, 2)$ : grad  $l_2 = (4, 8)$   
 point  $(3, 3)$ : grad  $l_3 = (2, 6)$   
 mean  $(4, 6.67) = \text{grad } L$   
 dots:  $\theta - 0.1$  grad for each pick; the batch step  $(3.6, -0.67)$  is their mean

$L = \frac{1}{n} \sum_i \ell_i$  and the gradient is linear, so  $\nabla L = \frac{1}{n} \sum_i \nabla \ell_i$ . For  $j$  drawn uniformly,

$$\mathbb{E}[\nabla \ell_j] = \sum_{i=1}^n \frac{1}{n} \nabla \ell_i = \nabla L.$$

Each  $\ell_i$  is its own valley with its own gradient, so one step may point the wrong way, but on average SGD points the right way.

## 9 Checked: unbiased, and a larger $b$ cuts the noise



The mean of 20,000 one-example gradients matches the full gradient to within 0.1%. The error of a batch of  $b$  distinct examples shrinks like  $\sqrt{(n-b)/(b(n-1))}$ , about  $1/\sqrt{b}$  for  $b \ll n$ , and is 0 at  $b = n$ .

## 12 Pause and think

One step on  $x^2 + 2$  from 2,  $\alpha = 0.1$ ?  $f' = 4$ , so  $x = 1.6$  and  $f$  falls from 6 to 4.56.

Does the normal equation always beat GD? No; it suits linear least squares with small  $d$ .

Is SGD always faster than batch GD? No. It is faster per epoch but noisy, and may need more epochs to settle.

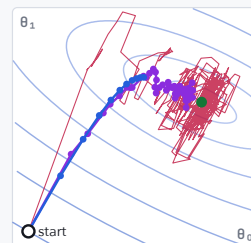
Does GD find the global minimum of any  $f$ ? No, a local one; global for convex  $f$  and small enough  $\alpha$ .

Should  $\alpha$  decay during training?

Often: a schedule lets SGD settle at the minimum.

Why standardise the features? It rounds the contours, so one  $\alpha$  suits every direction.

## 10 SGD: $n$ updates per epoch, on a noisy path

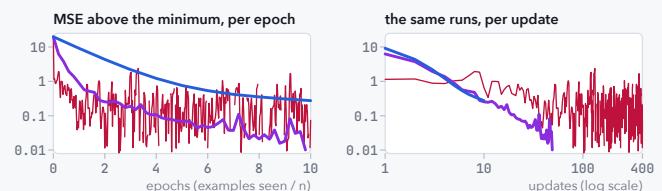


10 epochs each,  $\alpha = 0.05$ , same start data:  $y = 1 + 2x + \text{noise}$ , 40 points (ours)  
 — batch  
 10 updates, MSE 1.43  
 — mini-batch,  $b = 8$   
 50 updates, MSE 1.16  
 — SGD,  $b = 1$   
 400 updates, MSE 1.23  
 • least squares, MSE 1.15

Batch GD takes 10 smooth steps and is still on its way. SGD takes 400 and wanders near the minimum: with a fixed  $\alpha$  it never settles, and a decaying  $\alpha$  fixes that. The same noise can carry it out of shallow local minima of non-convex losses. Mini-batches of 8 take 50 steadier steps and end closest. One loop covers all three; this is the mini-batch run:

```
# b = n: batch GD; b = 1: SGD
for epoch in range(10):
    perm = rng.permutation(n)
    for k in range(0, n, b):
        i = perm[k:k + b]
        r = X[i] @ theta - y[i]
        g = 2 / len(i) * X[i].T @ r
        theta = theta - alpha * g
    print(theta.round(2)) # [1.05 2.18]
```

## 11 SGD wins per epoch, big batches per update



After one epoch SGD is 0.11 above the minimum MSE, batch GD 9.2. A GD iteration ( $X\theta$ , residual,  $X^T r$ ) costs  $O(nd)$ ,  $T$  of them  $O(Tnd)$ ; an SGD update  $O(d)$ . The normal equation costs  $O(d^2n + d^3)$  time and  $O(d^2)$  memory once: fine for  $d$  below about 1000, but most models have no closed form. Momentum, RMSprop and Adam, which damp the zigzag of panel 6, build on the same loop.

step along  $-\nabla L$  · keep  $\alpha < 2/\lambda_{\max}$  ·  $\mathbb{E}[\nabla \ell_j] = \nabla L$ : unbiased but noisy ·  $O(nd)$  per batch step,  $O(d)$  per SGD step

Before: the linear regression II sheet; next: the matrix factorisation sheet. Source: lecture slides "Gradient descent", the SGD notes, and the Taylor-series, gradient-descent, contour and cost-iteration notebooks ([nipunbatra.github.io/ml-teaching](https://nipunbatra.github.io/ml-teaching)). The 40-point regression data and all random draws are ours, computed with fixed seeds.